

Operating Systems — Homework 1

Exercise 1.1 – 1.4

Ex. 1.1 (5 pts) — Magic number of a binary executable

Role. A magic number is a short, fixed byte pattern that a file carries so that its *format* can be recognised from its contents rather than from its name. When the kernel services an `execve()`, it does not trust the file extension: it reads the first handful of bytes and matches them against the formats it knows, and that match is what selects the right loader (native ELF loader, script interpreter, emulation layer, ...). If nothing matches, the call is simply refused with `ENOEXEC` instead of the machine jumping into arbitrary bytes. It therefore does two jobs at once: format dispatch and a cheap integrity/sanity check.

Where it is stored. In the very first bytes of the file, at offset 0, as the opening field of the file header — which is exactly what makes the check cheap, since the kernel already has to read that first block anyway. Examples: an ELF binary starts with `0x7F 'E' 'L' 'F'` (the first four bytes of `e_ident` in the ELF header); a shell or Python script starts with `#! (0x23 0x21)` followed by the interpreter path; a Windows PE image starts with `'M' 'Z'`; a Java class file starts with `0xCAFEBAFE`.

Ex. 1.2 (5 pts) — Segments of an address space

A process address space is built from the following regions, conventionally laid out from low addresses to high:

- **Text (code) segment** — the machine instructions of the program. Mapped read-only and executable, so it can be shared by every process running the same binary and cannot be scribbled over by a stray pointer. Fixed size.
- **Initialized data segment (.data)** — global and static variables whose initial value is not zero. Their values are stored in the executable file and copied in by the loader. Read/write, fixed size.
- **Uninitialized data segment (BSS)** — global and static variables that start out zero. Only their *size* is recorded in the file, not their contents; the loader zero-fills the region, so the BSS costs nothing on disk. Read/write, fixed size.
- **Heap** — memory obtained at run time by `malloc/new`. It sits above the BSS and **grows upward**, toward higher addresses, as the allocator extends it (`brk/sbrk`, or `mmap` for large requests).
- **Memory-mapped region** — shared libraries, files mapped with `mmap`, and large allocations. It occupies the middle ground between heap and stack, and is how dynamically linked code is brought into the space.
- **Stack** — one per thread; holds activation records: parameters, local variables, saved registers and return addresses. It is placed at the top of the space and **grows downward**, toward lower addresses, one frame per call.

The heap and the stack thus grow toward each other, and the unmapped gap between them is what keeps them apart; a program that runs off either end walks into that hole, which is precisely the situation of Ex. 1.4.

Ex. 1.3 (5 pts) — Von Neumann machines and machines today

A von Neumann machine is the *stored-program* design: a CPU (control unit plus ALU and registers), a single main memory that holds instructions *and* data indistinguishably, and I/O, all joined by a shared bus. Its defining idea is that the program is itself just data sitting in memory, so the machine is general purpose — you change what it does by loading different bytes, not by rewiring it — and it operates by repeating the fetch–decode–execute cycle, with a program counter naming the next instruction. The price of the single shared memory and bus is that instruction fetch and data access compete for the same path, the well-known *von Neumann bottleneck*.

Today’s computers are still von Neumann machines at the level the programmer sees: x86, ARM and RISC-V all present one flat address space containing both code and data, a program counter, and sequential instruction semantics — which is why a compiled program can be written into memory and then jumped to at all. What has changed is everything underneath, added mostly to hide that bottleneck: split L1 instruction and data caches (a Harvard-style separation smuggled inside a von Neumann machine), deep pipelines, superscalar and out-of-order execution, branch prediction, and many cores sharing memory. So the model is inherited essentially unchanged; only the implementation has become elaborate.

Ex. 1.4 (5 pts) — “Segmentation fault (core dumped)”

What the message means. The program touched memory it was not entitled to touch. The MMU checks every address as it is translated, and when the address is either not mapped at all or is mapped with the wrong permission (writing a read-only page, executing a non-executable one), it raises a hardware fault. The kernel finds no legitimate reason for the fault, converts it into the signal `SIGSEGV`, and delivers it to the process; the default action for `SIGSEGV` is to kill the process and write out a core dump. The line you see is the shell afterwards reporting how its child died.

Possible reasons.

- Dereferencing a null pointer (typically an unchecked return from `malloc` or `fopen`).
- Dereferencing an uninitialized or otherwise wild pointer.
- Use-after-free, or following a dangling pointer to a stack frame that has already returned.
- Running past the end of an array or buffer, far enough to leave the mapping.
- Writing through a `char *` that points at a string literal, which lives on a read-only page.
- Unbounded recursion overflowing the stack into the guard page below it.
- Corrupting the allocator’s metadata (double free, overrun of a heap block), so that a later allocation returns nonsense.

What “segmentation” means here. The word is historical. On machines with segmented memory the address space was divided into segments, each with a base, a limit and access rights, and the hardware trapped any reference that fell outside the bounds or rights of the current segment — a *segmentation violation*. Mainstream systems are paged rather than segmented now, but the name stuck, and it means what it always meant: an access outside a valid, permitted region of the address space.

What “core” means here. “Core” is magnetic-core memory, the ferrite-ring main memory of machines of the 1950s and 60s; “core” became a synonym for main memory and outlived the technology. A core dump is therefore an image of the process’s memory — its segments as listed in Ex. 1.2 — together with its register state, captured at the instant it faulted and written to a file so that a debugger can be pointed at it afterwards and reconstruct where and why the program died.

Exercise 1.5 – 1.8

Ex. 1.5 (5 pts) — What a compiler may assume about loading

Why the compiler cannot know. A program is compiled once but run many times, on machines the compiler will never see. Where the binary ends up depends entirely on conditions that exist only at the moment of launch: how much physical memory the machine has, which other processes are already resident, and therefore which free hole the loader happens to pick. Nothing about that is available while the source is being translated. Modern systems push the answer even further away from compile time — address-space layout randomisation deliberately relocates the image on every run, shared libraries are mapped wherever there is room, and demand paging means a page is not given a physical frame until it is first touched. Two runs of the same binary on the same machine can therefore sit at different addresses, so no fixed address could have been correct anyway.

What it assumes instead. The compiler sidesteps the problem by generating code into a *logical* (virtual) address space that it assumes **begins at address 0** and is **contiguous**. Every internal reference — branch targets, function entry points, addresses of globals — is expressed as an offset from that assumed origin, so the whole image can later be shifted anywhere as a unit. The actual binding of logical to physical addresses is deferred to somebody better informed: either a relocating loader that patches the addresses when it places the image, or hardware (a relocation/base register and the MMU) that adds the offset on every reference at run time. This is exactly what makes *relocatable* and *position-independent* code possible. The three binding times are worth naming: *compile time* (absolute code, only usable if the load address is known in advance), *load time* (relocatable code, fixed up once when loaded), and *execution time* (bindings resolved on each reference, which is what lets a process be moved while running — and what essentially all general-purpose operating systems use today).

Ex. 1.6 (5 pts) — From double-click to window on screen

The two cases differ *only* in how the command line is assembled; once a path and an argument vector exist, the remaining path through the OS is identical. A double-click makes the desktop environment look up the icon’s file association or desktop entry to recover an executable path plus arguments; a typed command makes the shell tokenise the line, expand variables and globs, and resolve the command name against the directories listed in `PATH`. From there:

1. `fork()` (or an equivalent `spawn`) creates a child process, copy-on-write.
2. The child calls `exec()` on the resolved executable path.
3. The kernel checks permissions (execute bit, ownership) and opens the file.
4. It reads the header and validates the **magic number**, which identifies the binary format (ELF, PE, Mach-O) and selects the right loader; a bad magic number fails the `exec` rather

than running garbage.

5. The child's old address space is torn down and a fresh one constructed: text, initialised data, zero-filled BSS, heap and stack, with `argv/envp` copied onto the new stack.
6. The segments are set up by memory-mapping the file rather than reading it in, so almost no I/O happens yet.
7. If the binary is dynamically linked, the interpreter named in its header (`ld.so`) is mapped and runs first, mapping the required shared libraries and performing relocations and symbol resolution.
8. The kernel updates the process control block — page tables, inherited file descriptors, reset signal handlers — and puts the process on the ready queue.
9. The scheduler eventually dispatches it and execution starts at the entry point; pages are faulted in on demand as they are actually touched.
10. Runtime startup code initialises the language runtime and then calls `main()`.
11. The program connects to the window system (or inherits the terminal), creates a window and draws into it; the compositor puts those pixels on the screen.

The last difference is on the parent's side: a shell normally `wait()`s for the child (unless it was backgrounded), whereas a GUI launcher usually does not and returns to its event loop immediately.

Ex. 1.7 (5 pts) — Virtual to physical under contiguous allocation

Under contiguous allocation each process occupies one unbroken block of physical memory, which makes translation extremely cheap: the entire mapping is described by two registers that the kernel loads on a context switch, the **relocation (base) register** holding the block's starting physical address and the **limit register** holding its length. The MMU applies them in a fixed order — *check first, then add*:

$$\text{physical} = \begin{cases} \text{base} + \text{virtual}, & \text{if } \text{virtual} < \text{limit}, \\ \text{addressing trap to the OS}, & \text{if } \text{virtual} \geq \text{limit}. \end{cases} \quad (1)$$

For example, with `base = 14000` and `limit = 3000`:

$$\begin{aligned} \text{virtual } 346 : 346 < 3000 &\Rightarrow \text{physical} = 14000 + 346 = 14346, \\ \text{virtual } 3200 : 3200 \geq 3000 &\Rightarrow \text{addressing trap (no access)}. \end{aligned}$$

The process only ever names logical addresses in $[0, \text{limit})$ and never sees a physical address, so it is structurally incapable of referring to memory outside its own block — that is where the protection comes from. Loading the base and limit registers is a privileged instruction available only to the kernel, which is what stops a process from simply widening its own bounds.

Ex. 1.8 (5 pts) — The round-robin scheduler

Round robin is first-come-first-served made preemptive by a clock. The ready queue is treated as a circular FIFO and each process, on reaching the front, is given the CPU for at most one fixed **time quantum** q (typically tens of milliseconds); if it is still running when the timer interrupt fires, the OS preempts it, saves its context and moves it to the tail of the queue, while a process that blocks

or finishes early simply gives the CPU up sooner. With n runnable processes, every one of them therefore receives roughly $1/n$ of the CPU in slices no longer than q and waits no more than about $(n - 1)q$ for its next turn.

The choice of q is the whole design decision: too large and no process is ever actually preempted, so the algorithm degenerates into plain FCFS, while too small and the system spends a growing fraction of its time context-switching instead of doing work — q must stay large relative to the context-switch cost. Its virtues are fairness and a bounded, predictable response time with no possibility of starvation, which is why it suits interactive and time-sharing systems. Its vices are a comparatively poor average turnaround time (worse than SJF) and the fact that it is deliberately blind: it takes no account of priority, deadlines, or how much work a job actually has left.

Ex. 1.9 — Three Papers from “OS History and Architecture”

From the “OS History and Architecture” group of the paper list (papers 1-1 through 1-11) I read **Paper 1-10** (Dijkstra, “THE”), **Paper 1-2** (Ritchie and Thompson, UNIX) and **Paper 1-3** (Engler, Kaashoek and O’Toole, Exokernel). I chose these three because they form a single argument carried across nearly thirty years: structure by *layers*, then structure by *uniform abstraction*, then the proposal to *abolish* the abstraction altogether.

Paper 1-10 — Dijkstra, “The Structure of the “*THE*”-Multiprogramming System”

Summary. Dijkstra reports on a multiprogramming system built at Eindhoven for the Electrologica EL X8 — a machine with a $2.5\mu\text{s}$ core cycle, 27-bit words, 32K of core and a 512K-word drum — by six people of, on average, half-time availability. The goals are deliberately modest: shorter turnaround for short jobs, economical use of peripherals and automatic control of backing store. He is explicit that this is *not* a multi-access system; users share only the configuration and an ALGOL 60 procedure library.

Two structural ideas carry the design. The first decouples information from storage. “Segments” (units of information) are named independently of “pages” (units of memory, whether core or drum), and a segment variable in core records where a segment currently lives. A segment evicted to the drum therefore need not return to the drum page it came from, so the system simply picks the free page with minimum rotational latency and the drum allocation problem disappears entirely.

The second is the level hierarchy. The system is a society of sequential processes running at undefined speed ratios, coordinated only by explicit **P** and **V** operations on semaphores; the appendix defines these and separates semaphores used for mutual exclusion from *private* semaphores, on which only the owning process ever performs a P. The processes are stacked in a strict hierarchy: level 0 allocates the processor and absorbs the real-time clock; level 1 is the segment controller and the drum bookkeeping; level 2 is the message interpreter owning the console keyboard; level 3 buffers the peripherals; level 4 holds user programs; level 5 is the operator, “not implemented by us”. Each level implements an abstraction that makes the level below invisible — above 0 the number of processors is irrelevant, above 1 pages have lost their identity, above 2 every process appears to own a private console.

The claimed payoff is verification, not speed. Because a process can only generate tasks for processes at lower levels, circularity is excluded; harmonious cooperation is then proved in three stages, ending

in the absence of circular waits — what Dijkstra calls “the Deadly Embrace”. Testing followed the same order, level 0 first and a new level added only once the one beneath it was exhausted. The reported outcome is that only trivial coding errors appeared, at a density of one per 500 instructions, each located within ten minutes.

Critical judgment. The case for layering here is purely structural, and the paper contains essentially no performance data — we are never told what the hierarchy *cost*, in memory, in context switches, or in the latency added by pushing every I/O request down through three levels. The most revealing sentence in the paper is the concession that at the time of writing testing was not yet complete, “but the resulting system is guaranteed to be flawless”. That is a claim about the rigour of a design being offered in place of evidence about an artifact, and the two are not the same thing. Worse, it is in tension with Dijkstra’s own report a paragraph earlier: what the proof covers is the synchronisation skeleton, whereas the errors he actually found were coding errors — exactly the class the proof cannot reach. He is transparent about both facts without ever conceding that the second undermines the first.

The strict ordering of levels also shows a stress crack that the paper itself records. Keeping the hierarchy acyclic required a special precaution: a segment requested from the drum must be pinned in core until the requesting process has actually touched it, since otherwise one finite task could spawn unboundedly many requests on the segment controller, and the system would thrash. This is the level discipline failing, by itself, to deliver the property it was introduced to guarantee: acyclicity bounds which processes may be asked to do work, but not how often, so termination had to be rescued by a pinning rule that lives outside the hierarchy and is justified nowhere in it. It also hints at why strict total ordering did not survive — in later systems the pager needs the disk driver, which needs memory, which needs the pager. Layering endured as a discipline; the total order did not.

Finally, the scope does a great deal of quiet work in the argument. Six people, one known configuration, no multi-access, no user-written machine code, and no security model at all is a setting in which exhaustive testing is plausible in a way it is not elsewhere. Dijkstra anticipates precisely this objection — industry doubts the technique scales — and answers that “the larger the project, the more essential the structuring”. This may well be true, but as stated it is rhetoric rather than evidence, and it is the one claim in the paper with no support behind it. What did survive, and survived enormously, is the deeper thesis: that one should choose a structure *because* it makes correctness provable. That line runs directly from here to seL4.

Paper 1-2 — Ritchie and Thompson, “The UNIX Time-Sharing System”

Summary. This describes the third version of UNIX, on the PDP-11/40 and /45. The scale is part of the argument: a 16-bit machine with 144K bytes of core of which UNIX occupies 42K, hardware costing as little as 40,000 dollars, and less than two man-years spent on the main system software, with about 40 installations running since February 1971. The system had been rewritten in C during the summer of 1973; the C version is about a third larger than the assembler one, which the authors accept because it is far easier to understand and modify and brought multiprogramming and shared reentrant code with it.

The technical content is a small set of uniform abstractions. There are three kinds of file — ordinary, directory, and special. Ordinary files are unstructured strings of bytes: structure is the business of the programs that use a file, not of the system. Every device appears as a special file in `/dev`, which

the authors argue has a threefold advantage — file and device I/O become as similar as possible, file and device names share one syntax, and devices fall under the same protection mechanism as files. Directories form a rooted tree with the conventional `.` and `..` entries, and all links to a file have equal status, because a directory entry holds nothing but a name and a pointer to an i-node. The i-node carries eight device addresses; small files use them directly and large files indirect through blocks of 256 addresses, which caps a file at $8 \cdot 256 \cdot 512 = 2^{20}$ bytes. `mount` grafts a removable volume's hierarchy onto a leaf of the existing one.

Processes are equally spare. `fork` splits a process into two, `execute` overlays the image with a new program, and `wait/exit` join them again; `pipe` returns a descriptor for an interprocess channel that is read and written with the same `read` and `write` calls as a file. The Shell is not part of the kernel at all but an ordinary swappable user program that forks a process per command. Because the Shell — not the command — interprets `<` and `>`, no command contains any code for redirection; filters joined by `|` compose for the same reason. `init` creates one process per terminal line, and since a user's password-file entry may name some other program in place of the Shell, the same mechanism confines the editing-only users and the users who may only play chess. Section 9 gives the operational picture: 72 users, 14 simultaneous, 4400 files, and per day 1800 commands, 4.3 CPU hours and 75 logins, at about 98% uptime with a longest unbroken run of roughly two weeks.

Critical judgment. The paper is unusually honest, and that honesty is simultaneously its charm and its weakness as an evaluation. The entire performance argument is section 4.1: one assembly of a 7621-line program in 35.9 seconds, or 212 lines per second, split 63.5% assembler, 16.5% system overhead, 20% disk wait — immediately followed by the statement that the authors will not interpret these figures or compare them with any other system, and are “generally satisfied”. A design paper is entitled to that stance, but the consequence is that every efficiency claim in the paper rests on assertion. The famous “Perspective” remark, that UNIX succeeded largely because it was not designed to meet any predefined objectives, is charming and probably true, but it is unfalsifiable and useless as advice — one cannot plan to have no requirements.

The treatment of security is thin, and the mechanisms it introduces so casually became the classic problems. Protection is a user ID plus seven bits: read, write and execute for the owner and for everyone else — there is no group in this version — plus set-user-ID, plus a super-user exempt from all checking. Set-user-ID is presented purely as an elegance, available to any user on his own files without administrative intervention. The paper never asks what happens when such a program can be induced to act on its invoker's behalf, and decades of privilege-escalation bugs live in that unexamined gap.

“Everything is a file” is also less total than the slogan it became, and the authors say so where it is inconvenient. Pipes are conceded to be “not a completely general mechanism” because a common ancestor must create them — the reason named pipes and then sockets had to be bolted on later, and the first place the uniform interface visibly leaks. The 1 MB file ceiling and the 14-character name limit are not principled choices but artifacts of the i-node layout. The accounting problem created by equal-status links is raised and then dropped with the admission that UNIX avoids the issue by charging no fees at all. There is essentially nothing on multiprocessing. And section 9 describes a research laboratory, not a production load — chess alone accounts for 5.3% of CPU time — so the reliability figures cannot support conclusions about UNIX under commercial pressure. What the paper does establish, and establishes decisively, is a negative result: the expense and bulk of contemporary operating systems were not intrinsic to the problem.

Paper 1-3 — Engler, Kaashoek and O’Toole, “Exokernel”

Summary. The target is the fixed abstraction itself. Any abstraction the kernel imposes — virtual memory, IPC — embodies a trade-off, between sparse and dense address spaces, or read-intensive and write-intensive access, and whichever way it is resolved some class of application pays. The authors cite measurements by Cao et al. showing that application-controlled file caching alone can cut running time by 45%. Their response is to separate protection from management: the kernel should securely multiplex the raw hardware and nothing more, while *library operating systems*, linked into the application’s own address space and untrusted, implement processes, virtual memory and IPC. Changing the operating system becomes relinking.

Three mechanisms make this safe. *Secure bindings* decouple authorisation from use: the kernel performs the expensive check once, when a binding is established, and afterwards enforces it cheaply. They are implemented by hardware support, by software caching — a large software TLB acting as a cache of frequently used bindings — or by downloading code into the kernel, of which packet filters are the paradigm case, application-supplied predicates that demultiplex packets without the owning application being scheduled. *Visible revocation* makes the library a participant rather than a victim: even the CPU is taken back by an explicit revocation when a time slice ends, so a library that knows its floating-point state is dead can decline to save it. When a library will not cooperate, the *abort protocol* breaks the bindings by force, records the seizure in a *repossession vector* and raises an exception, with a small guaranteed reserve of pages so the library can survive being repossessed.

The prototype is Aegis, which exports the processor, physical memory, TLB, exceptions, interrupts and — via a packet filter built with dynamic code generation — the network; and ExOS, which implements processes, virtual memory, user-level exceptions, IPC and ARP/RARP, IP, UDP and NFS. Both run on MIPS DECstations and are compared against Ultrix 4.2. The headline numbers: exception dispatch in $1.5\ \mu\text{s}$ on a DECstation5000/125, roughly five times the best published figure and about two orders of magnitude faster than Ultrix; protected control transfer nearly seven times the best reported implementation; the dynamic packet filter classifying TCP/IP headers in $1.5\ \mu\text{s}$ against $35\ \mu\text{s}$ for MPF; and an application-level stride scheduler in under 100 lines of code. The authors conclude that the architecture is viable for high-performance extensible systems.

Critical judgment. The diagnosis is excellent and the evaluation cannot carry it. Nearly every number is a microbenchmark of a single primitive, obtained by repeating the operation many times and averaging, which — as the authors state themselves — means the measurements “do not consider cold start misses in the cache or TLB, and therefore represent a best case”. A system whose entire premise is that real applications are being held back by the kernel is evaluated without a real application. The one whole-program measurement, a 150×150 matrix multiplication, is dominated by the compiler and says almost nothing about the operating system underneath it.

The comparison is also structurally unfair in a way the paper concedes in a single sentence and then walks past: ExOS does “not offer the same level of functionality as Ultrix”, and “we do not expect these additions to cause large increases in our timing measurements”. That expectation is exactly the claim that needed evidence, because the absent functionality — swapping, and a real file system still described as under development — is precisely where a general-purpose system spends its money. The admission that the prototype “has no real users” should be read alongside the ten-to-100 $\times$ speedups: a factor measured against a system that is doing strictly more work is a much weaker result than the number suggests.

The security argument is asserted more than demonstrated. Secure bindings are supposed to preserve protection while removing management, but no adversary model is developed and no attack is

attempted; the paper notes in passing that one must ensure a packet filter does not “lie” and accept packets destined for another application, and leaves the problem there. There is also a deep tension the authors do not resolve: exposing physical names and physical resources to untrusted software makes the interface hardware-specific by construction, which is the portability cost of the whole design, and their own dismissal of VM/370 — that virtualising the base machine is expensive — was overtaken within a decade by hardware virtualisation that made that cost largely vanish. History’s verdict is therefore split, and that is the fair judgment: the critique of fixed abstractions was right and has outlived the artifact, while the deployment argument was wrong. Library operating systems became practical roughly twenty years later, as unikernels and VM-hosted specialisation — which is to say the idea won on someone else’s mechanism.

The Three Together

Read in sequence these papers are one long argument about where structure should live. Dijkstra puts it in the kernel as a total order of levels, and buys provability with rigidity. Ritchie and Thompson put it in a handful of uniform abstractions — the byte stream, the i-node, the special file, the shell as an ordinary program — and buy extraordinary leverage at the price of leaks wherever the uniformity is false. Engler and his co-authors argue that any such fixed abstraction is a tax, and push structure out of the kernel entirely into replaceable libraries.

The pattern in their evidence is at least as instructive as the pattern in their designs. Dijkstra offers a proof and no measurements; Ritchie and Thompson offer one measurement they decline to interpret; the exokernel offers a great many measurements, all of primitives and none of a workload anyone actually ran. Each paper is strongest exactly where the previous one was weakest and weakest in a new place — which is a reasonable summary of how the field made progress.

References

- [1-10] E. W. Dijkstra. “The Structure of the “*THE*”-Multiprogramming System.” *Communications of the ACM*, vol. 11, no. 5, May 1968, pp. 341–346. (Presented at the ACM Symposium on Operating System Principles, Gatlinburg, Tennessee, October 1967; also circulated as EWD196.)
- [1-2] D. M. Ritchie and K. Thompson. “The UNIX Time-Sharing System.” *Communications of the ACM*, vol. 17, no. 7, July 1974, pp. 365–375. (Revised version of a paper presented at the Fourth ACM Symposium on Operating Systems Principles, Yorktown Heights, New York, October 1973.)
- [1-3] D. R. Engler, M. F. Kaashoek and J. O’Toole Jr. “Exokernel: An Operating System Architecture for Application-Level Resource Management.” *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP ’95)*, 1995, pp. 251–266.

Ex. 1.0 — Three Papers on OS History and Architecture

The three papers below were located by searching the literature (Google Scholar and general web search for work on operating system history and architecture), downloaded in full and read; none of them is taken from the supplied paper list. They were chosen as a set because they form a

single argument across forty-four years. Each gives a different answer to the same two questions: *where should operating system function live*, and *what may the system assume about the machine underneath it?* Multics answers with one comprehensive system on a symmetric shared-memory machine; Mach answers by decomposing that system into a minimal kernel plus user-level servers; the Multikernel answers by discarding shared memory as the organising assumption altogether. Read in order, they also show a steady rise in evidentiary standards — from a design prospectus with no measurements at all, to a status report resting on a single microbenchmark, to a broad evaluation that volunteers its own weaknesses — without any matching fall in the confidence of the architectural claim being made.

Paper 1 — Corbató and Vyssotsky, “Introduction and Overview of the Multics System” (1965)

Summary. The paper opens the six-paper Multics session at the 1965 Fall Joint Computer Conference and states the objectives the other five then elaborate. Its problem is access: batch processing had, in the authors’ words, caused user access to large machines to “retrogress,” isolating the programmer from cause and effect. Their goal is a *computer utility* — a system running continuously and reliably twenty-four hours a day like a telephone or power network, serving everything from interactive terminals to absentee (batch) jobs. The mechanisms proposed are largely hardware–software co-design on the GE 645, a modification of the GE 635. Addressing is two-dimensional: programs are written as *segments*, each of which may grow or shrink during execution (up to roughly a quarter of a million segments per user, each of up to a quarter of a million words), and segments are independently *paged* by the supervisor with a choice of 64- or 1024-word pages. Cross-segment references are bound by *dynamic linking* — a segment names another only symbolically, binding occurs automatically on first execution, and afterwards runs at full speed. Pure, recursive, shareable procedures are the normal mode of programming, so that one copy of a procedure serves many users and, as the authors note, there is “no clear-cut demarcation” between user and system programs. Protection comes from descriptor bits acting as hardware “fire-walls,” including an execute-only bit, and most supervisor modules run with the same descriptors as user code and no access to privileged instructions. Processors form an anonymous pool with no master — “the supervisor does not have a special processor” — and reach I/O only through mailboxes in the memory modules. A file system names everything symbolically rather than by address, interlocks writing, and provides system-run backup against three named contingencies. I/O is device-independent, batch is simply the zero-terminal case of an ordinary process, resource budgets are hierarchical and delegable, PL/I is adopted for machine independence, and the reference manual is kept on-line. There is no evaluation: the system did not yet exist. The sole quantitative claim is that “a few hundred” simultaneous users are expected, derived by “simple scaling of processor and memory speed” and immediately qualified as “hazardous.” The paper closes by calling the plans “not unattainable” while conceding it would be “presumptuous” to think the initial system could meet all the stated requirements.

Critical judgment. This is a prospectus, and it cannot be judged as an empirical paper: every verb is in the future tense and nothing in it is falsifiable by its own evidence. The deeper omission is not measurement but a *cost* estimate. The paper proposes to meet “almost all of the present and near-future requirements” of a large installation and never once asks what such generality would cost to build. That is precisely where the project failed — development ran years late, Bell Labs withdrew in 1969, and UNIX was written shortly afterwards as a deliberate reaction against exactly this scale. The most accurate sentences in the paper are its own hedges. Elsewhere it argues rhetorically rather

than evidentially: “machine-aided cognition,” the promised social consequences, and above all the claim that PL/I would deliver machine independence are asserted rather than demonstrated, and the last of these badly underestimated the cost of building the compiler. On substance, history has split the paper cleanly in two. Vindicated: paged virtual memory, device-independent I/O, a symbolically named hierarchical file system with per-file access control and system-provided backup, symmetric multiprocessing with no master processor, writing an operating system in a high-level language, and graded hardware protection — the descriptor bits became Multics’ rings, which x86 inherited. Refuted: user-visible *segmentation* lost decisively to the flat, paged address space, and x86-64 all but abolished segmentation; nobody today programs a two-dimensional address space, and although dynamic linking survived it did so as lazy load-time binding over a flat space, not as segment binding. The most interesting case is the computer utility itself. The economic argument — that bulk economies, floor space and management efficiency favour one maximal centralised machine — was falsified for roughly thirty years by the minicomputer and the personal computer, then vindicated by cloud computing. The paper was right about the destination and wrong about both the timing and the reason, since what eventually made centralisation win was statistical multiplexing at datacentre scale, not the considerations it actually advanced.

Paper 2 — Accetta et al., “Mach: A New Kernel Foundation for UNIX Development” (1986)

Summary. Mach’s target is the accretion of mechanism in UNIX. The authors observe that a system which began with a small uniform set of operations on file descriptors had grown “a staggering number” of overlapping facilities — System V streams, 4.2BSD sockets, ptys, several kinds of semaphore, shared memory, and a “mind-boggling array” of `ioctl` operations — giving non-uniform access to resources both within a machine and across a network, while the process abstraction remained too heavyweight to express parallelism. Mach replaces the base with four abstractions. A *task* is the unit of resource allocation, owning a paged address space together with whatever port rights it holds; a *thread* is the unit of CPU utilisation, so that a UNIX process is simply a task with a single thread; a *port* is a kernel-protected message queue acting as a capability, with any number of senders but exactly one receiver; and a *message* is a typed collection of data that may carry port rights and may be as large as an entire address space. Every operation on any other object is a message to the port representing it, so that the kernel is merely one server among many — the design thesis is that “the actual system running on any particular machine is a function of its servers rather than its kernel.” The virtual memory system supports per-page inheritance (shared, copy, or none) and a current-versus-maximum protection pair in which the maximum may only ever be lowered; copy-on-write implements both `fork` and bulk message transfer, illustrated with a 24 MB message that passes through a temporary kernel map. Most consequentially, page faults may be serviced *outside* the kernel by user-level pagers, so that a memory-mapped file is simply memory whose pager is the file system, and the default pager uses ordinary file systems rather than a paging partition. The implementation separates machine-dependent from machine-independent structure behind a bare validate/invalidate/protect interface — the independent page size is fixed at boot as a power-of-two multiple of the machine-dependent one, itself 512 bytes on the VAX — letting the same machine-independent code sit above VAX page tables and the RT/PC’s inverted page table. Matchmaker, an interface definition language, generates RPC stubs for C, Pascal and CommonLisp, and user-level network servers extend ports across machines transparently. As of April 1986 Mach was binary compatible with 4.3BSD and running on assorted VAXen, a four-processor VAX 11/784 and the IBM RT/PC.

Critical judgment. The gap between the ambition of the design and the weight of the evidence is very wide, and the paper is candid that “extensive performance comparisons with other systems have not yet been done.” In practice its entire quantitative evaluation is a single number: on a MicroVAX II, touching newly allocated memory costs under 0.7 ms per 1024 bytes against roughly 1.2 ms for 4.3BSD. That microbenchmark measures zero-fill fault cost — exactly what the new VM system was built to improve — and was chosen by the implementers; `fork` is called “substantially faster” with no figure attached at all. Nothing measures IPC latency, although ports and messages are the centrepiece of the design; nothing measures multiprocessor scaling, although the abstract’s opening sentence announces a “multiprocessor operating system kernel”; threads, the feature the paper leads with, were not yet implemented and were merely “expected by Summer 1986”; and the caption of the paper’s own final figure concedes that UNIX compatibility still executed in kernel state. The central claim — that operating system services can be moved out to user-level servers at acceptable cost — is therefore precisely the claim the paper does not test.

History then split the verdict. The mechanisms won comprehensively: separating the address space from the schedulable entity is now universal, memory-mapped files and external pagers are standard equipment, the machine-dependent/independent VM split survives as the `pmap` layer in the BSDs, and port-like capability handles reappear in Windows object handles and in Fuchsia’s channels, while macOS and iOS run on a Mach derivative to this day. The performance thesis lost: Mach 3.0’s user-space UNIX server proved slow, Chen and Bershad later traced the degradation to memory-system behaviour rather than to message path length, and Liedtke’s L4 work argued the cost was an artefact of implementation rather than of microkernels, cutting IPC by roughly an order of magnitude. Mach became the cautionary tale that motivated second-generation microkernels.

Two of its assumptions have since expired outright. Network-transparent IPC treats a local multiprocessor and a LAN as points on one continuum, an equivalence the distributed systems community abandoned because transparency conceals latency and partial failure. And copy-on-write as the universal bulk-transfer mechanism assumed page-table manipulation was cheap relative to copying, which inverted once TLB shutdown across many cores entered the price. Finally, the extensibility argument is made by *enumerating* UNIX’s accumulated mess rather than by showing that the port model resists the same accretion — and it did not: Mach’s own VM, with its chains of shadow objects, became notoriously intricate, and the kernel that carries its name today is large.

Paper 3 — Baumann et al., “The Multikernel: A New OS Architecture for Scalable Multicore Systems” (2009)

Summary. The Multikernel’s premise is that hardware now diversifies faster than system software can be retuned for it, so that an OS built as a shared-memory kernel with lock-protected data structures faces an unbounded engineering bill. The authors evidence this with the “heroic” removal of the Windows 7 dispatcher lock, which touched 6000 lines across 58 files, and with the many iterations required by Linux’s read-copy-update. They then argue that the machine has itself become a network — interconnects route and congest, cores differ in ISA and performance, and peripherals are already non-coherent — and propose three principles: make all inter-core communication explicit, make OS structure hardware-neutral, and treat state as replicated rather than shared. Their prototype, Barrelfish, factors the per-core OS instance into a privileged *CPU driver* (event-driven, single-threaded, non-preemptable, 7135 lines of C for x86-64) and a schedulable user-space *monitor*. Monitors hold the replicas and run the agreement protocols — one-phase commit for unmapping a page, two-phase commit for capability retying — over URPC, a user-level message

transport tuned to the cache-coherence protocol, in which the sender writes a cache line sequentially and the receiver polls its last word. Memory is managed by seL4-style capabilities with all page-table manipulation performed in user space, and a system knowledge base holding hardware facts in first-order logic computes topology-aware multicast trees at run time. The motivating measurement is stark: a single core updates shared state in under 30 cycles, but with 16 cores on the same data each update costs roughly 12,000 extra cycles, and one dedicated RPC server sustains twice the update rate of all 16 shared-memory threads together. The evaluation then shows a NUMA-aware multicast shutdown protocol holding at about 3.1k cycles across 32 cores where naive broadcast and unicast climb to roughly 13.8k and 12.7k respectively; end-to-end page unmapping at 32 cores costing about 17.5k cycles against roughly 29.5k for Linux 2.6.26 and 46k for Windows Server 2008 R2; IP loopback at 2154 versus 1823 Mbit/s with 21 versus 77 data-cache misses per packet; UDP echo over gigabit Ethernet at 951.7 against 951 Mbit/s; and a static web server at 18,697 requests per second against 8924 for lighttpd on Linux.

Critical judgment. This is much the strongest evaluation of the three and an unusually honest one — it states outright that “it would be wrong to draw any quantitative conclusions from our large-scale benchmarks,” concedes that the capability system was “a mistake... unnecessarily complex, and no more efficient” than the per-processor allocators already in Windows and Linux, calls its network stack “very much a placeholder,” and admits that every test platform was homogeneous x86-64, so the heterogeneity that motivates the whole argument was never actually exercised. That honesty makes the weak points easier to locate. The headline numbers are the least defensible: the $2.1\times$ web-server win pits a purpose-built user-space lwIP stack against general-purpose lighttpd on Linux with segmentation and checksum offload enabled, while Barrelfish’s own driver supported none of it, so what is measured is specialisation rather than architecture.

The scalability case rests on global-coordination microbenchmarks, and there the gain comes largely from replacing serial IPIs with a topology-aware multicast *tree* — an algorithmic change a shared-memory kernel could equally adopt — so the experiment does not isolate shared-nothing structure as the cause of the improvement. Reading the shutdown and unmap figures against each other is more revealing still: the raw messaging baseline costs about 3.1k cycles at 32 cores while the complete unmap costs about 17.5k, so more than four-fifths of the operation is Barrelfish’s own monitor LRPC, marshaling and unoptimised thread dispatch. The shape of the unmap curve makes the same point at the other end. Barrelfish does not start cheaper than the systems it is compared against — at two cores it costs about 10.3k cycles against roughly 5.2k for Linux and 5.8k for Windows — and it stays almost perfectly flat out to five cores before rising gently. What it buys is slope: from two to thirty-two cores Barrelfish grows about $1.7\times$, reaching some 17.5k cycles, where Linux grows $5.7\times$ to about 29.5k and Windows $7.9\times$ to about 46k. So the multikernel did not reduce the constant cost of the operation; it roughly *doubled* it and bought a much flatter curve in exchange. That is exactly the trade the authors themselves concede when they note that routing OS invocations through monitor RPC rather than system calls adds “a constant overhead on current hardware of several thousand cycles,” which is “constant as the number of cores increases” — a bargain that pays only once the machine is wide enough, and whose break-even against Linux does not arrive until somewhere around thirteen or fourteen cores. The messaging table quietly reinforces the point, showing URPC at 450 cycles of latency against 424 for L4’s decade-older IPC, the real gains again being throughput and a much smaller cache footprint rather than latency.

The hardware bet was half right. Cache coherence did not collapse — commodity servers remained coherent far past 2009 — and mainstream kernels absorbed NUMA and core asymmetry using per-CPU data, RCU and sharded locks, settling in the middle of the paper’s own spectrum of

sharing disciplines rather than at the shared-nothing extreme it advocates. Barrelfish remained a research vehicle. What survived is the diagnosis and the method: treating the machine as a network, which resurfaced in kernel-bypass and dataplane systems such as Arrakis and IX, and in the now-routine practice of designing OS state per-core first. One irony deserves noting — having adopted the distributed systems analogy, the paper declines its principal prize, stating that “the complete system is a failure unit,” so it pays the full price of messaging and replication without buying any tolerance of partial failure.

Comparing the Three

Placed side by side, the three papers are three attempts to answer the same question — what may the operating system take for granted about the machine? — and each is undone at a different layer. Multics assumed that generality could be centralised in one comprehensive system, and was defeated by the complexity budget it never wrote down. Mach assumed the boundary could be pushed outward so that services became user-level servers, and was defeated by the cost of crossing that boundary. The Multikernel assumed shared memory was itself the wrong primitive, and was overtaken by shared-memory hardware that went on scaling for another decade. Their standards of proof move in the opposite direction to their confidence: 1965 offers a design prospectus with no measurements, 1986 a status report resting on one microbenchmark of the subsystem its authors had just optimised, and 2009 a broad evaluation that explicitly marks its own limits — yet the boldness of the architectural claim does not diminish as the evidence improves, which is a useful caution when reading any systems paper. Most striking is that all three have been vindicated in mechanism while being refuted in scope. Multics’ paging, protection rings and device independence are everywhere, but its segmented address space is gone. Mach’s tasks, threads, ports and memory objects are everywhere, but its argument that services could be evicted from the kernel for free is not. The multikernel’s per-core replication now sits inside Linux, but as an optimisation within a shared-memory kernel rather than as a replacement for one. The durable contribution of an architecture paper, on this evidence, is rarely the architecture.

References

- [1] F. J. Corbató and V. A. Vyssotsky. “Introduction and Overview of the Multics System.” In *Proceedings of the AFIPS Fall Joint Computer Conference (FJCC)*, vol. 27, part 1, pp. 185–196, 1965.
- [2] M. Accetta, R. Baron, W. Bolosky, D. Golub, R. Rashid, A. Tevanian and M. Young. “Mach: A New Kernel Foundation for UNIX Development.” In *Proceedings of the USENIX Summer Conference*, pp. 93–112, 1986.
- [3] A. Baumann, P. Barham, P.-E. Dagand, T. Harris, R. Isaacs, S. Peter, T. Roscoe, A. Schüpbach and A. Singhanian. “The Multikernel: A New OS Architecture for Scalable Multicore Systems.” In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pp. 29–44, 2009.